

Unit IV: Design

Software design is the bridge between requirements and coding. After gathering and analyzing user requirements, developers prepare a design (blueprint) of the software before writing the actual code. A good design makes software easy to understand, develop, test, maintain, and improve.

Why is Software Design Important?

- Converts requirements into a software blueprint.
- Helps developers understand how the system will be built.
- Reduces coding errors and development cost.
- Makes testing and maintenance easier.
- Improves software quality, reliability, and performance.

Main Activities in Software Design

1. Designing the data.
2. Designing the software architecture.
3. Designing the user interface.
4. Designing individual components.
5. Preparing the software for deployment.

Design Concepts

1. Design Process

Definition

The **Design Process** is the step in the Software Development Life Cycle (SDLC) where software requirements are converted into a detailed design or blueprint for implementation.

The design process is the activity of transforming software requirements into a detailed plan that guides coding and testing.

Objectives

- Convert requirements into design.
- Improve software quality.
- Reduce development cost.
- Make software easy to maintain.
- Support future changes.

Steps of the Design Process

1. Study software requirements.
2. Create the overall system architecture.
3. Design data structures.
4. Design user interfaces.
5. Design software components.
6. Review and improve the design.
7. Start coding based on the approved design.

Characteristics of a Good Design

- Simple and easy to understand.
- Modular.
- Flexible.
- Reusable.
- Efficient.
- Easy to test and maintain.

Advantages

- Reduces coding mistakes.
- Saves development time.
- Improves software quality.
- Simplifies testing.
- Makes maintenance easier.

Example

While developing an **Online Shopping System**, before coding developers design:

- Product database
- Payment process
- User interface
- Order processing modules

Only after this design is complete does coding begin.

Design Process: The process of converting software requirements into a detailed software blueprint before coding.

2. Design Model

Definition

A **Design Model** is a representation or blueprint of the software system. It shows **how the software will be organized, how different parts will interact, and how the system will be implemented** before coding starts.

A Design Model is a blueprint that describes the structure and behavior of a software system before implementation.

Why is a Design Model Needed?

- Converts requirements into a clear design.
- Helps developers understand the system.
- Reduces coding errors.
- Improves communication among team members.
- Makes testing and maintenance easier.

Elements of a Design Model

A design model generally consists of the following parts:

1. Data Design

- Defines how data is organized and stored.
- Includes database tables, files, and data structures.

2. Architectural Design

- Shows the overall structure of the software.
- Divides the system into different modules.

3. Interface Design

- Defines interaction between the user and the system.
- Also describes communication between software modules.

4. Component-Level Design

- Describes the internal logic of each module.
- Explains algorithms and functions.

5. Deployment Design

- Shows how software will be installed and executed on hardware.

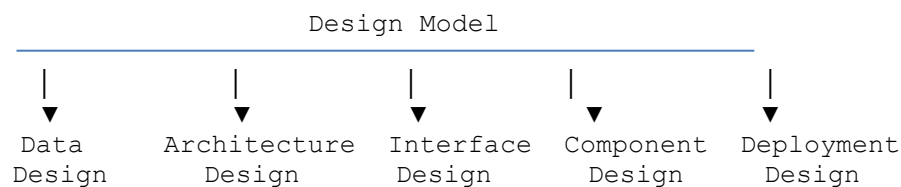
Characteristics of a Good Design Model

- Easy to understand.
- Simple and modular.
- Flexible for future changes.
- Reusable.
- Reliable.
- Easy to test and maintain.

Advantages

- Provides a clear software blueprint.
- Reduces development time.
- Improves software quality.
- Makes coding easier.
- Simplifies maintenance.

Diagram



Example

Online Shopping System

- **Data Design:** Customer, Product, Order tables.
- **Architecture Design:** Login Module, Cart Module, Payment Module.
- **Interface Design:** Login page, Product page, Payment page.
- **Component Design:** Order processing algorithm.
- **Deployment Design:** Web server, Database server, Client browser.

Architecture Design Model

Introduction

The **Architecture Design Model** describes the **overall structure of a software system**. It divides the software into different modules (components) and shows how these modules communicate with each other.

It is also called the **high-level design** because it focuses on the entire system rather than individual modules.

Definition

Architecture Design Model is the process of defining the overall structure of a software system by dividing it into modules and specifying their relationships and interactions.

Objectives

- Divide the system into modules.
- Improve maintainability.
- Increase software reusability.
- Make the system easy to understand.
- Improve performance and scalability.

Components of Architecture Design

1. Modules

Independent parts of the software.

Example:

- Login Module
- Payment Module
- Report Module

2. Connectors

Communication links between modules.

3. Interfaces

Rules through which modules exchange information.

4. Data Flow

Shows how data moves from one module to another.

Types of Software Architecture

- Layered Architecture
- Client-Server Architecture
- MVC (Model-View-Controller)
- Microservices Architecture

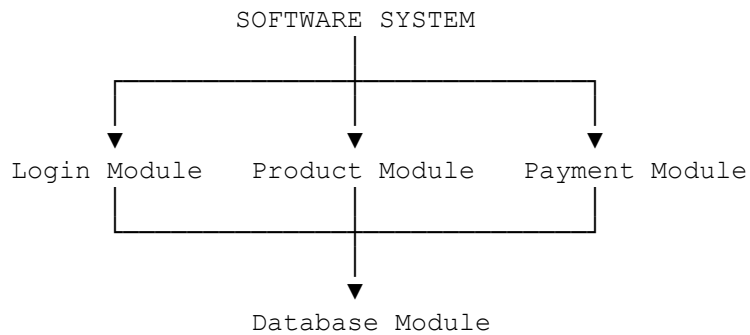
Characteristics of Good Architecture

- Modular
- Scalable
- Reliable
- Secure
- Easy to maintain

Advantages

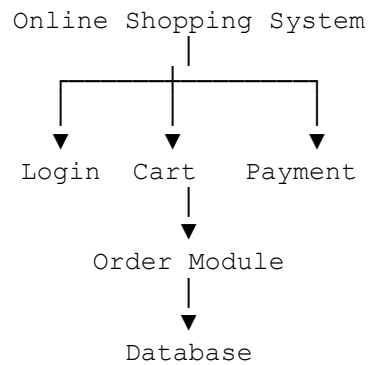
- Easy maintenance
- Better teamwork
- Easy testing
- Reusable modules
- Supports future expansion

Diagram



Example

Online Shopping System



Transform Mapping and Transaction Flow

Introduction

After designing the software architecture, the next step is to decide **how data moves inside the software** and **how different user requests are processed**.

There are two important design concepts:

1. **Transform Mapping**
2. **Transaction Flow**

These help developers convert requirements into a proper software structure.

A. Transform Mapping

Definition

Transform Mapping is a design technique used when data enters the system, is processed, and then produces output.

Transform Mapping is the process of converting input data into output through a series of processing steps.

Stages of Transform Mapping

1. Input Flow

- Data enters the system.
- Example: User enters login details.

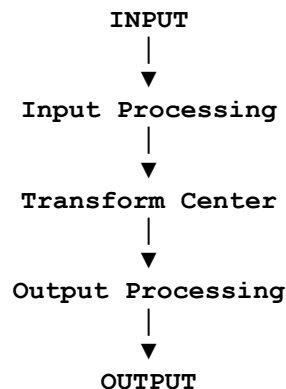
2. Transform Center

- Data is processed.
- Example: Username and password are verified.

3. Output Flow

- Result is displayed.
- Example: Login successful or invalid password.

Diagram



Example

Student Result System

Input:

- Roll Number

↓

Processing:

- Search marks
- Calculate total
- Calculate percentage

↓

Output:

- Display Result

Advantages

- Easy to understand.
- Suitable for sequential processing.
- Makes software modular.
- Easy to test and maintain.

B. Transaction Flow

Definition

A Transaction Flow is used when one input can lead to different processing paths, depending on the user's choice or request.

Transaction Flow is a design model in which one input selects one of many possible processing paths.

Components

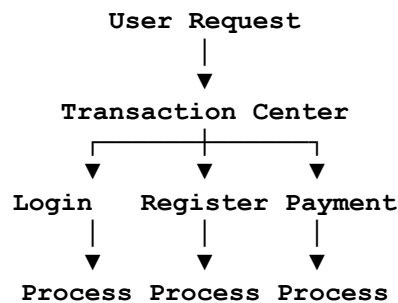
1. Transaction Center

- Receives the user's request.
- Decides which path to follow.

2. Action Paths

- Different modules perform different tasks.

Diagram



Example

ATM Machine

Customer inserts ATM card.

The system shows options:

- Withdraw Money
- Deposit Money
- Balance Inquiry
- Mini Statement

Each option follows a different processing path.

Advantages

- Easy to handle multiple user requests.
- Improves software organization.
- Easy to add new functions.

- Supports modular design.

Difference Between Transform Mapping and Transaction Flow

Transform Mapping	Transaction Flow
Data flows in a sequence.	User chooses one of many paths.
Single processing flow.	Multiple processing flows.
Used for continuous data processing.	Used for menu-driven systems.
Example: Result calculation.	Example: ATM, Online Banking.

Interface Design

Introduction

Interface Design is the process of designing **how users interact with the software** and **how different software modules communicate with each other**. A good interface should be simple, attractive, consistent, and easy to use.

Interface design focuses on creating an effective communication between the user and the software system.

Definition

Interface Design is the process of designing the interaction between the user and the software system so that it is easy, efficient, and user-friendly.

Objectives of Interface Design

- Make the software easy to use.
- Improve user experience.
- Reduce user errors.
- Ensure consistency in screens.
- Make navigation simple.

Principles of Interface Design

1. Simplicity

- Keep the interface simple and easy to understand.

2. Consistency

- Use the same style, colors, buttons, and menus throughout the software.

3. User-Friendly

- The interface should be easy for all users.

4. Feedback

- The system should respond to every user action.
- Example: "Login Successful" or "Invalid Password".

5. Error Handling

- Show clear error messages.
- Help users correct mistakes.

Components of Interface Design

1. Input Interface

- Accepts data from the user.
- Example: Login Form, Registration Form.

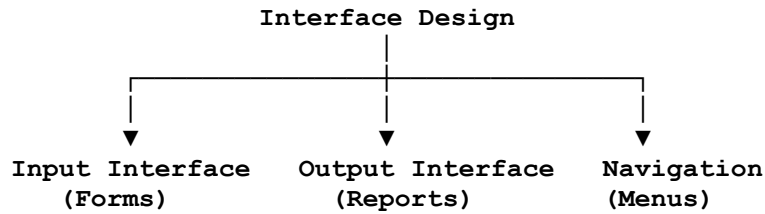
2. Output Interface

- Displays information to the user.
- Example: Report, Result, Receipt.

3. Navigation Interface

- Helps users move between screens.
- Example: Menu, Home Button, Back Button.

Diagram



Example

Online Shopping Website

- **Input Interface:** Login page, Search box.
- **Output Interface:** Product details, Order confirmation.
- **Navigation Interface:** Home, Cart, Payment, Profile.

Advantages

- Easy to learn and use.
- Reduces user mistakes.
- Improves customer satisfaction.
- Saves time.
- Increases software usability.

Disadvantages

- Good interface design requires time.
- Poor design can confuse users.
- Frequent redesign increases development cost.

Component-Level Design Elements

Component-Level Design describes the **internal working of each software component/module**. It explains what a particular module does and how it performs its task.

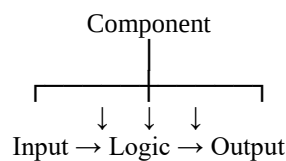
Definition

Component-Level Design is the detailed design of individual software components, including their functions, logic, inputs, outputs, and interactions.

Main Elements

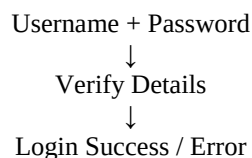
1. **Component/Module** – A separate part of the software that performs a specific task.
2. **Interface** – Defines how the component communicates with other components.
3. **Input** – Data required by the component.
4. **Processing/Logic** – Operations performed by the component.
5. **Output** – Result produced by the component.

Diagram



Example

Login Component:



Advantages

- Makes coding easier.
- Makes testing easier.
- Helps identify errors.
- Improves maintenance.
- Allows components to be reused.

Component-Level Design:

It explains the **internal details and working of individual software modules**.

Deployment-Level Design Elements

Deployment-Level Design describes **where and how the software will be installed and run** on the actual hardware or computing environment.

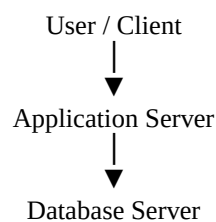
Definition

Deployment-Level Design defines the **physical arrangement of software components on hardware devices or servers.**

Main Elements

1. **Hardware** – Computers, servers, mobile devices, etc.
2. **Software Components** – Applications, modules, databases, services.
3. **Network** – Communication between different devices.
4. **Server** – Runs and provides software services.
5. **Client** – Device through which the user accesses the software.

Diagram



Example

For an **Online Shopping System**:

- User → Mobile/Computer
- Application → Web/Application Server
- Data → Database Server

Advantages

- Shows where software components will run.
- Helps in system installation.
- Improves performance planning.
- Helps manage network communication.
- Makes system maintenance easier.

Deployment-Level Design:

It shows **how software components are placed and executed on physical hardware and servers.**

Design Principles:

1. Abstraction

Abstraction means **showing only the important information and hiding unnecessary details.**

Definition

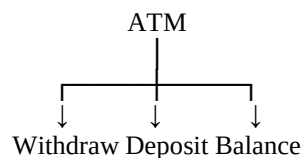
Abstraction is the process of identifying the essential features of a system while hiding unnecessary implementation details.

Types of Abstraction

1. **Data Abstraction**
Shows important data and hides how it is stored internally.
2. **Procedural Abstraction**
Shows what a function does without showing its internal steps.
3. **Control Abstraction**
Hides complex control operations behind simple instructions.

Example

When we use an **ATM**:



The user sees these options, but the internal banking process is hidden.

Advantages

- Reduces complexity.
- Makes software easier to understand.
- Improves maintainability.
- Helps developers focus on important features.

Abstraction = Show important details + Hide unnecessary details.

2. Architecture

Architecture means the **overall structure of a software system**. It shows how the major parts/modules of software are arranged and connected.

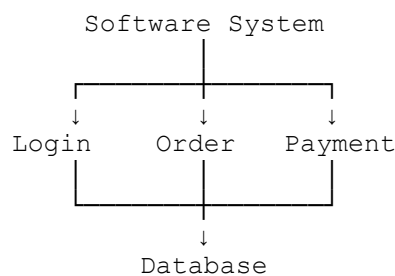
Definition

Software Architecture is the **high-level structure of a software system that defines its major components and their relationships**.

Main Elements

1. **Components** – Major parts/modules of the software.
2. **Connectors** – Show communication between components.
3. **Interfaces** – Define how components interact.

Simple Diagram



Example

In an **Online Shopping System**:

- Login → manages users
- Order → manages orders
- Payment → handles payments
- Database → stores information

Advantages

- Gives a clear view of the complete system.
- Makes the system easier to understand.
- Helps in maintenance and modification.
- Supports scalability and reuse.

Architecture = Overall structure of the software and relationships between its major components.

3. Design Patterns

Design Patterns are **reusable solutions to common software design problems**. They provide a standard way to solve frequently occurring design issues and improve software quality.

Definition

A design pattern is a proven and reusable solution to a recurring software design problem.

Types

1. Creational Patterns

Creational patterns deal with **object creation**. They provide flexible ways to create objects and reduce dependency on specific classes.

Examples: Singleton, Factory Method, Abstract Factory, Builder, Prototype.

2. Structural Patterns

Structural patterns deal with the **organization of classes and objects**. They define how different classes and objects are combined to form larger and more flexible structures.

Examples: Adapter, Decorator, Facade, Composite, Proxy.

3. Behavioral Patterns

Behavioral patterns deal with **communication and interaction between objects**. They define how objects work together and share responsibilities efficiently.

Examples: Observer, Strategy, Command, State, Iterator.

Example

Singleton Pattern ensures that **only one object** of a class exists throughout the application.

Example: Printer Manager or Database Connection.

Advantages

- Provides reusable solutions.
- Improves software quality.
- Makes software easier to maintain.

- Saves development time.
- Promotes code reusability.

Disadvantages

- Wrong pattern selection can increase complexity.
- Learning different patterns takes time.
- Not necessary for very small applications.

Design Pattern = Reusable solution for common software design problems.

4. Modularity

Modularity means **dividing a software system into small, independent modules**, where each module performs a specific function.

Definition

Modularity is the process of dividing a software system into separate modules, each responsible for a specific task.

Types

1. Functional Modularity

The software is divided into modules based on different **functions or tasks**.

Example: Login Module, Payment Module, Report Module.

2. Object-Oriented Modularity

The software is divided into **classes and objects**, where each class represents a specific entity.

Example: Student Class, Employee Class, Product Class.

Example

A **Library Management System** is divided into separate modules such as:

- Book Management
- Student Management
- Issue/Return Module
- Fine Calculation Module

Each module performs its own task independently.

Advantages

- Reduces software complexity.
- Makes testing easier.
- Simplifies maintenance.
- Encourages code reuse.
- Makes development faster.

Disadvantages

- Too many modules may increase management effort.
- Improper module design can reduce performance.

Modularity = Divide software into small, independent modules for easier development and maintenance.

5. Functional Independence

Functional Independence means **each module performs one specific task and has minimum dependency on other modules**. It is achieved through **High Cohesion** and **Low Coupling**.

Definition

Functional Independence is the property of a software module that performs a single function with minimal interaction with other modules.

Components

1. High Cohesion

High cohesion means **all functions within a module are closely related and work together to perform a single task**.

Example: A Login Module handles only user authentication.

2. Low Coupling

Low coupling means **modules have minimum dependency on each other**, so changes in one module have little effect on others.

Example: The Payment Module works independently of the Student Module.

Example

In an **Online Shopping System**:

- Login Module → User authentication
- Product Module → Product management
- Payment Module → Payment processing

Each module performs its own task with minimum dependency on other modules.

Advantages

- Easy to understand.
- Easy to test and debug.
- Simplifies maintenance.
- Improves code reusability.
- Changes in one module have minimal impact on others.

Disadvantages

- Designing highly independent modules requires careful planning.
- Initial design may take more time.

Functional Independence = High Cohesion + Low Coupling

6. Refinement

Refinement means **breaking a large and complex problem into smaller, simpler, and manageable parts**. It is also called **Stepwise Refinement**.

Definition

Refinement is the process of gradually developing a software design by breaking it into smaller and more detailed components.

Types

1. Top-Down Refinement

The design starts from the **main system** and is gradually divided into smaller modules until each module is simple to implement.

Example: Library System → Book Management → Issue Book → Update Database.

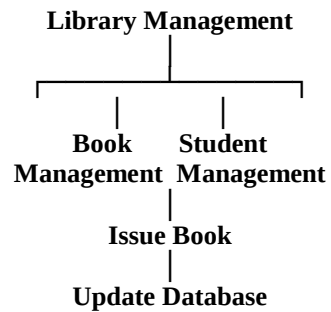
2. Stepwise Refinement

The design is developed **step by step**, adding more details at each stage until the complete solution is obtained.

Example: Online Shopping → Payment Module → Card Payment → OTP Verification.

Example

Library Management System



Advantages

- Reduces software complexity.
- Makes the design easy to understand.
- Improves software quality.
- Simplifies coding and testing.
- Makes maintenance easier.

Disadvantages

- Takes more planning time.
- Poor refinement can lead to unnecessary modules.

Refinement = Break a large problem into smaller, manageable parts (Top-Down approach).

7. Refactoring

Refactoring means **improving the internal structure of existing code without changing its external behavior or functionality**. It makes the code cleaner, easier to understand, and easier to maintain.

Definition

Refactoring is the process of restructuring existing code to improve its readability, maintainability, and quality without changing its functionality.

Common Refactoring Techniques

1. Rename Method

Changes the name of a method or variable to make its purpose clearer.

Example: Rename `calc()` to `calculateTotal()`.

2. Extract Method

Moves a block of code into a separate method to improve readability and reuse.

Example: A long `processOrder()` function is divided into `validateOrder()` and `calculateBill()`.

3. Remove Duplicate Code

Eliminates repeated code by creating a common method or function.

Example: Common login validation code is placed in a single method instead of repeating it.

Example

Before Refactoring:

One long function with repeated code.

After Refactoring:

Small, well-named functions with no duplicate code.

Advantages

- Improves code readability.
- Makes maintenance easier.
- Reduces duplicate code.
- Simplifies debugging and testing.
- Improves software quality.

Disadvantages

- Requires careful testing after refactoring.
- Can take extra development time.

Refactoring = Improve code quality without changing its functionality.

8. Object-Oriented Design (OOD) Concepts

Object-Oriented Design (OOD) is a design approach based on **objects and classes**.

Definition

OOD is the process of designing software using classes, objects, and object-oriented principles.

Main Concepts

1. Class
2. Object
3. Encapsulation
4. Inheritance
5. Polymorphism
6. Abstraction

Example

A **Car** class can create objects like **BMW**, **Audi**, and **Honda**.

Advantages

- Code reuse.
- Easy maintenance.
- Better security.
- Easy scalability.

OOD = Designing software using classes and objects.

9. CASE Tools in Software Design

CASE (Computer-Aided Software Engineering) Tools are software applications that help developers **design, develop, test, and maintain software.**

Definition

CASE tools are software tools used to automate software development activities.

Types

1. Upper CASE Tools
2. Lower CASE Tools
3. Integrated CASE Tools

Example

- StarUML
- Visual Paradigm
- IBM Rational Rose

Advantages

- Increases productivity.
- Reduces development time.
- Improves documentation.
- Reduces errors.

CASE Tools = Software tools that automate software development tasks.